Contents

CLAUDIU user manual

CLAUDIU (working codename) is a local, browser-based tabbed interface for running several Claude Code sessions at once. A small Python server spawns real `claude` CLI processes in pseudo-terminals and streams them into browser tabs via xterm.js, so every tab behaves exactly like a real terminal — same colors, same prompts, same keybindings — while the browser layer adds tabs, a calmer theme, adjustable fonts, snippets, and a project launcher on top.

Quick start

```
pip install -e .
python -m claudiu
```

or, on Windows, double-click `run_claudiu.bat`. The server starts on `http://127.0.0.1:8642/` (127.0.0.1 only — never reachable from other machines) and opens it in your default browser automatically.

Features

- **Tabs** — one tab per live `claude` session, named after the project folder and renamable.
- **Terminal fidelity** — each tab embeds a real interactive `claude` process rendered by xterm.js: skills, plugins, slash commands, and permission prompts all behave exactly as in a plain terminal.
- **Session survival on refresh** — sessions live on the server, not in the browser tab; refreshing or closing Chrome never kills them.
- **Reconnect with backoff** — a dropped websocket retries automatically with backoff behind a visible "reconnecting…" strip; it never fails silently.
- **Resume after crash** — after a browser or PC crash, a resume list offers one-click `claude --resume <session-id>` per recent project.
- **Snippets bar + palette** — frequently used prompt text as buttons under the tab strip and in a fuzzy-searchable palette (`Ctrl+k`).
- **Project launcher** — configured working folders (plus a browse option) in the new-session dialog; one click starts `claude` in the chosen directory.
- **Per-tab font size** — each tab's font size is adjustable and remembered independently.
- **Scrollback search** — search within a tab's capped scrollback, with highlighting.
- **Rename by double-click** — double-click a tab's label to rename it.
- **Status strip** — above each terminal, a one-line box shows the last prompt you typed in that session (click it to expand long prompts). Each tab carries a small light — pulsing amber while Claude is working, a faster accent-coloured pulse when it is waiting for your input on a permission prompt, and steady green once it is done — and a context gauge that fills as the conversation grows and shifts from green through amber to red as it approaches the point where auto-compaction starts hurting. A tab whose turn finishes

while you are looking elsewhere gets the "unseen" dot. All three read Claude Code's own session transcript; nothing is guessed from terminal output (see "How the status strip works" below).

- **Light / dark / system theme** — a theme button in the tab bar cycles the interface between following your OS (`system`), `light`, and `dark`; the choice is remembered per browser. Set the default with `ui_theme`. (The terminal keeps its own `theme` colours in every mode.)
- **Conversation view (controlled)** — by default a session shows a clean, CLAUDIU-rendered conversation instead of Claude Code's raw terminal: the working directory and Claude's window title on top, the exchange in a scrollable frame (PageUp/PageDown, a jump-to-latest button, collapsible tool output and thinking), lane checkboxes (**thinking · tools · events · subagents**) and a search box, the model and token/context count, the live state, and a composer to type into. It is read from Claude Code's transcript JSON and refreshed about once a second — stable, no reflow. A permission prompt appears as **buttons** matching the prompt's real options (parsed from the rendered screen, shown only when unambiguous; otherwise CLAUDIU tells you to open the terminal rather than guess). When you need a menu the view can't model (slash-command autocomplete, `/model`, the plan or file pickers), **Show terminal** reveals the real terminal, and hides it again. See "The conversation view" below.
- **Redesigned launcher** — the new-session window opens with "Any folder" on top (type a path, pick a recently used one from the dropdown, or **Browse** the filesystem and **New folder** to create one), and two independently scrolling columns below: your configured **Projects** and **Resume recent**.
- **Jump to latest** — when you scroll up in a tab, a "↓ latest" button appears to return to the live output.
- **Calm dark theme** — near-black background, warm-gray foreground, muted ANSI colors, steady non-blinking cursor; every color is a config value. Sessions start with Claude Code's `dark-ansi` theme so *all* of its colours come from that palette (Claude Code's default themes print hard-coded truecolor that no terminal palette can soften); bold text keeps its colour instead of jumping to the bright variant.

Configuration

One human-editable JSON file. Default location `~/.claudiu/config.json`, created with defaults the first time the server runs if it does not exist yet. Override the path with the `CLAUDIU_CONFIG` environment variable, or with the `--config` flag on the command line. Keys the file does not mention fall back to the defaults below; keys the file has that this version of CLAUDIU does not recognize are kept as-is and reported as a startup warning, never rejected.

Key	Default	Meaning
`port`	`8642`	TCP port the server listens on (bound to 127.0.0.1 only)
`claude_command`	`["claude"]`	argv used to spawn each session
`replay_chunks`	`2000`	max buffered output *chunks* per session kept for replay on (re)connect (each chunk up to 64 KiB; see below)
`scrollback_lines`	`10000`	scrollback cap shown in the browser terminal
`font_size`	`16`	default terminal font size in pixels
`font_family`	`"Consolas, 'Cascadia Mono', monospace"`	terminal font stack
`theme`	a 16-color dark theme object	background, foreground, cursor, selection background, and the 16 ANSI colors
`claude_theme`	`"dark-ansi"`	Claude Code theme passed to every spawned session via `--settings`; `""` leaves Claude Code's own setting alone (its default themes bypass `theme` with truecolor)
`status_poll_ms`	`2000`	how often the page refreshes the status strip, light and gauge

Key	Default	Meaning
`context_window_tokens`	`200000`	the model's context window, denominator of the context gauge
`context_warn_pct`	`50`	gauge turns amber at this percentage of the context window
`context_danger_pct`	`75`	gauge turns red at this percentage (must be `context_warn_pct`)
`ui_theme`	`"system"`	interface theme: `"system"`, `"light"`, or `"dark"` (the tab-bar button overrides this per browser)
`permission_patterns`	four common prompts	substrings that mark a session as "waiting for you" when they appear in its terminal output (ANSI stripped, case-insensitive)
`recent_max`	`15`	how many recently launched folders to remember
`shortcuts`	see the Shortcuts table below	interface keyboard shortcut map
`projects`	`[]`	launcher entries: `{"name", "path", "args": []}`
`snippets`	`[]`	snippet-bar entries: `{"name", "text", "send": false}`

`replay_chunks` caps the number of buffered output *chunks*, not bytes or lines: each chunk is up to 64 KiB (the pty is read in 64 KiB blocks), so the worst-case memory a single session's replay buffer can hold is `replay_chunks` × `64 KiB` — 2000 × 64 KiB 125 MiB at the default. Raise or lower `replay_chunks` to trade replay depth against memory per session.

The conversation view

Each session shows two things over the same live process:

1. the **conversation view** (default) — rendered by CLAUDIU from the session's transcript JSON, refreshed ~1 s. It never shows Claude Code's raw terminal drawing, so it does not reflow or flicker. Tool output and thinking are collapsible; the lane checkboxes hide whole categories; the search box filters; PageUp/PageDown and a jump-to-latest button move through it. The composer at the bottom sends what you type to the session, and a detected permission prompt is offered as buttons.
2. the **raw terminal** — hidden by default, revealed by **Show terminal**. Use it for the interactive surfaces the JSON cannot represent (slash-command autocomplete, `/model`, the plan selector, the `@`-file picker, arrow-key menus). It is the same live session; toggling the view changes nothing about it.

Every keystroke CLAUDIU sends to a session (composer, a permission button, Esc, snippets) is written to the audit log (`~/.claudiu/logs/`, logger `claudiu.audit`) so there is a complete record of what was issued. If a Claude Code update adds a transcript record type CLAUDIU does not recognise, the header shows a small "unrecognized record(s)" warning instead of quietly dropping it.

Updates are polled, not streamed token-by-token — a reply appears when the turn lands, which is the calm behaviour this view is for. If a future Claude Code release changes the transcript schema the view degrades to empty rather than wrong, and the raw terminal is always one click away.

How the status strip works

Claude Code writes every session's transcript to `~/.claude/projects/<project-slug>/<session-id>.jsonl` as it runs. CLAUDIU starts each session with an explicit `--session-id`, so it knows which file belongs to which tab, and reads only the *tail* of that file (a few hundred KB at most) every `status_poll_ms`:

- **last prompt** — the transcript's `last-prompt` record (or the last user message you typed, never text injected by skills or hooks);

- **busy / ready / waiting** — the last assistant record's `stop_reason`: a finished turn means ready; a pending tool call, or a prompt with no answer yet, means busy. When the session is busy *and* its terminal shows one of the `permission_patterns` (a "Do you want to proceed?" prompt), the state becomes **waiting** — the transcript has no record for a blocked prompt, so this one signal is read from the terminal output rather than the transcript;
- **context gauge** — the last assistant record's token usage (input + cache-creation + cache-read), divided by `context_window_tokens`.

Sessions started with `--continue` have no id CLAUDIU can know in advance; their tab shows a dim light and no gauge. Sessions resumed with `--resume <id>` use that id. If the transcript cannot be found or read, the strip stays empty and the light stays dim — the terminal itself is never affected.

Worked example: adding a project and a snippet

Edit `~/.claudiu/config.json` (or the file `CLAUDIU_CONFIG` points at) and add to the `projects` and `snippets` lists:

```
{
  "projects": [
    {"name": "My App", "path": "C:/code/my-app", "args": ["--continue"]}
  ],
  "snippets": [
    {"name": "run tests", "text": "run the test suite and fix any failures",
     "send": true}
  ]
}
```

Restart the server to pick up the change (config is loaded once at startup; refreshing the page alone does not reload it). "My App" now appears in the project launcher, and "run tests" appears as a button in the snippets bar and in the `Ctrl+k` palette; because `send` is `true`, choosing it types the text and presses Enter.

Shortcuts

Interface-level keyboard shortcuts, all remappable via the `shortcuts` config key. They never intercept keys the CLI itself needs (plain `Ctrl+C`, `Ctrl+R`, arrow keys, and so on).

Action	Default key
`tab_1`	`Alt+1` — switch to tab 1
`tab_2`	`Alt+2` — switch to tab 2
`tab_3`	`Alt+3` — switch to tab 3
`tab_4`	`Alt+4` — switch to tab 4
`tab_5`	`Alt+5` — switch to tab 5
`tab_6`	`Alt+6` — switch to tab 6
`tab_7`	`Alt+7` — switch to tab 7
`tab_8`	`Alt+8` — switch to tab 8
`tab_9`	`Alt+9` — switch to tab 9
`tab_prev`	`Alt+ArrowLeft` — previous tab
`tab_next`	`Alt+ArrowRight` — next tab
`new_session`	`Alt+t` — open the new-session launcher
`close_tab`	`Alt+w` — close the active tab (asks; killing the session needs explicit confirmation)
`font_bigger`	`Alt+=` — increase this tab's font size
`font_smaller`	`Alt+-` — decrease this tab's font size
`font_reset`	`Alt+0` — reset this tab's font size
`search`	`Ctrl+Shift+f` — search this tab's scrollback
`snippet_palette`	`Ctrl+k` — open the fuzzy-searchable snippet palette

Action	Default key
help	Alt+h — show the shortcut list (also the ? button in the tab bar)

You do not need this table open while working: the ? button at the right of the tab bar (or Alt+h) shows the live bindings — including any you remapped — every tab's tooltip names its switch key, and the new-session launcher's footer repeats the two you need most.

Why Alt-based defaults: Chrome reserves Ctrl+T, Ctrl+W, Ctrl+Tab, and Ctrl+1…Ctrl+9 at the browser level — a web page cannot intercept them — so none of those combinations can ever be a working interface default; CLAUDIU's defaults use Alt instead. For the same reason, do not rebind any shortcut to bare Escape: the shortcut handler runs in the capture phase and would suppress the dialog-closing Escape handler that the launcher and rename dialogs rely on.

REST/WS API

Route	Description
GET /api/sessions	list live sessions
POST /api/sessions	create a session (body: path, args, title)
PATCH /api/sessions/<id>	rename a session (body: title)
DELETE /api/sessions/<id>	kill a session
GET /api/config	effective config plus load warnings
GET /api/resume	recent resumable Claude sessions
GET /api/status	per-session status: last prompt, busy/ready, context use
GET /api/conversation	controlled conversation model for a session (query: id)
GET /api/recent	recently launched folders (~/.claudiu/recent.json)
GET /api/dirs	list sub-directories of a path (launcher folder picker)
POST /api/mkdir	create a folder (body: parent, name)
WS /ws/<id>	terminal stream (terminado protocol)

GET /api/resume scans the real ~/.claude/projects directory of the user running the server — the resume list you see is your own Claude Code history on that machine, in that account. Each project entry includes an exists flag (true/false) so the launcher can skip projects whose directory has since moved or been deleted.

Command line

```
python -m claudiu [--config PATH] [--port PORT] [--log-dir DIR]
                  [--no-browser] [--verbose | --quiet] [--version]
```

Flag	Help
--config	path to config.json (default: ~/.claudiu/config.json, or $CLAUDIU_CONFIG)
--port	override the config port (config default: 8642)
--log-dir	directory for audit logs (default: <config dir>/logs)
--no-browser	do not open the browser automatically
--verbose	debug output on the console
--quiet	warnings and errors only on the console
--version	print the version and exit
--help	print the argument summary and exit

Logs

Every run writes an audit log to `<config dir>/logs/claudiu-<timestamp>.log` (override the directory with `--log-dir`) recording: the command line invoked, CLAUDIU/Python/Tornado/Terminado versions, any config load warnings, session start/rename/kill events, and how the run ended. Console verbosity is independent of the log file: `--verbose` prints debug lines to the console too, `--quiet` restricts the console to warnings and errors; the log file itself is always written at debug level.

Rebuilding this manual

This document is the source of truth; `USER_MANUAL.html` (and `USER_MANUAL.pdf`, when the tools are available) are built from it and committed so readers need no tooling of their own. To regenerate them after editing this file:

```
python docs/build_manual.py
```

Uses `pandoc` for the HTML and `pandoc` + `xelatex` for the PDF when both are on `PATH`; otherwise falls back to a small stdlib-only Markdown renderer for the HTML and prints that the PDF was skipped. Always exits 0. `--src` and `--outdir` override the input file and output directory.

Launching from inside Claude Code

If you start `claudiu` from a Claude Code session (a shell tool, a hook, a sub-agent), the server inherits that session's environment, and every `claude` it spawned would otherwise inherit it too — the CLI then treats itself as a nested child session and turns transcript saving off ("Transcript saving is off — inherited CLAUDE_CODE_CHILD_SESSION marker"). CLAUDIU strips the nesting markers (`CLAUDECODE`, `CLAUDE_CODE_CHILD_SESSION`, `CLAUDE_CODE_SESSION_ID`, `CLAUDE_PID`, the messaging/bridge/entrypoint/execpath variables) from each session's environment, so sessions started here are ordinary top-level sessions with normal transcript saving and `--resume` behaviour. User configuration such as `CLAUDE_CODE_MAX_OUTPUT_TOKENS` is passed through untouched.

Known limitations

- **Windows-first.** Development and hand-testing happen on Windows. Linux and macOS are exercised only by CI (unit, integration, and end-to-end tests all run there too) — they are not hand-tested.
- **One browser page per server.** The server assumes a single open page; it is not designed for multiple browser tabs/windows pointed at the same running instance.
- **No split panes.** One session per interface tab; no multi-pane layouts.
- **No remote access.** The server binds `127.0.0.1` only — it is not reachable from other devices, and there is no plan to make it so. Requests must also originate from the page itself: a foreign `Origin` or `Host` header (CSRF, or DNS rebinding to 127.0.0.1) is refused with 403.
- **No transcript export.** CLAUDIU does not render or export session transcripts; that is the job of the separate claude-session-publisher tool.
- **`localStorage` use is minimal.** The only thing CLAUDIU stores in the browser's `localStorage` is each tab's remembered font size — nothing else about a session lives in the browser; the browser is a disposable view onto server-side sessions.
- **The status strip is read from Claude Code's transcript files.** It depends on their current record shapes (`last-prompt`, `stop_reason`, `usage`); a future Claude Code release that changes them degrades the strip to "unknown" (dim light, no gauge, no prompt) — never to a wrong answer, and never to a broken terminal.
- **Keystrokes during a reconnect gap are dropped.** If you type while a tab's websocket is reconnecting (the "reconnecting…" strip is visible), those keystrokes are not queued and are lost — wait for the strip to clear before typing.